Pourquoi apprendre ceci ? Présentation

Génial ! Vous êtes en train de lire ce livre. Ce qui vous place dans l'une de ces trois catégories : un étudiant forcé d'étudier ce truc pour un cours, quelqu'un qui a décroché ce livre par pur accident (probablement parce que vous devez encore être endoctriné par le reste du monde), l'un des originaux qui ont encore de l'intérêt à étudier l'assembleur.

Oui-da. Quel genre de livre pourrait-on commencer de la sorte ? Quel genre d'auteur entamerait le livre avec un tel avant-propos ? Bon, la vérité est que j'avais songé à le placer dans le premier chapitre, vu que nombre de gens ne s'inquiètent pas de lire les avant-propos. Une discussion sur ce qui est juste et sur ce qui est faux à propos de l'assembleur est très importante et la glisser dans un chapitre pourrait encourager quelqu'un à la lire. Cependant, j'ai rapidement réalisé que les étudiants universitaires peuvent sauter le premier chapitre aussi bien que l'avant-propos ; par conséquent, j'ai enfin décidé de placer ce dernier à sa juste place.

Alors et encore, pourquoi apprendre l'assembleur ? Des suppositions possibles :

• Votre spécialisation demande ce type de cours ; c'est-à-dire vous êtes probablement ici contre votre volonté.

• Un programmeur, là où vous travaillez vient de démissionner. Il a laissé beaucoup de code source écrit en assembleur et vous avez la charge de le maintenir.

• Votre chef a eu la hardiesse d'insister pour vous faire écrire votre code en assembleur, contre vos souhaits.

• Vos programmes sont tout simplement trop lents ou tout simplement trop volumineux et vous pensez que l'assembleur pourrait vous aider à remettre votre projet sous contrôle.

• Vous voulez comprendre comment les ordinateurs fonctionnent.

• Vous avez intérêt à écrire du code efficace.

• Vous voulez essayer quelque chose de nouveau.

Quelle que soit la raison, vous êtes ici, bienvenue à bord. Prenons une vision du sujet.

1 Ce qui est faux au sujet de l'assembleur

Le langage assembleur a une petite mauvaise réputation. L'impression commune sur les programmeurs en assembleur est qu'ils sont tous des hackers ou des individus malavisés qui ont besoin d'un éclaircissement. Voici les raisons que les gens se donnent pour *ne pas* utiliser l'assembleur[1] :

• L'assembleur est dur à apprendre.

• L'assembleur est difficile à lire et à comprendre.

• L'assembleur est difficile à déboguer.

• L'assembleur est difficile à maintenir.

• L'assembleur est difficile à écrire.

• La programmation assembleur demande trop de temps.

• Des compilateurs améliorés ont éliminé le besoin de l'assembleur.

• De nos jours, les machines sont tellement rapides qu'on n'en a plus tellement besoin.

• Si vous voulez plus de vitesse, utilisez plutôt un meilleur algorithme.

• Les ordinateurs ont tellement de mémoire aujourd'hui qu'économiser de l'espace à l'aide de l'assembleur n'a plus d'importance.

• Le langage assembleur est, par définition, non portable.

[1] Ce texte utilisera les termes "langage assembleur" et "assembleur" interchangeablement.

Certaines de ces affirmations sont totalement fausses !

Etant donné que ce livre enseigne la programmation assembleur, qu'il a été écrit pour des étudiants d'université et par quelqu'un qui, en toute apparence, sait de ce dont il parle, votre tendance naturelle serait de croire à ce qu'on écrit ici. Après avoir lu ces lignes on dirait que l'assembleur est sans espoir. Et ceci, cher ami, correspond au quatre-vingt pourcent de ce qui est faux. Cela dit, les gens développent de bien fausses conceptions, des idées basées sur ce qu'ils ont entendu dire par des amis, instructeurs, des articles ou des livres. Certes, l'assembleur n'est pas parfait. Il a même des défauts évidents. Et cependant, ces lacunes sont gonflées tout autrement par ceux qui en fait ne connaissent pas ce langage. La prochaine fois que quelqu'un vous donne une opinion au sujet, demandez-lui : "combien d'années d'expérience avez-vous en assembleur ?" Certainement, l'assembleur est difficile à comprendre. *Pour ceux qui ne le connaissent pas.* Et c'est surprenant comme beaucoup de gens le démolissent en se basant uniquement sur des conversations ou des lectures d'articles.

Les programmeurs assembleur se servent aussi des langages de haut niveau (LHN)[2]. Alors que ceux qui s'opposent à l'assembleur n'utilisent que ces derniers. Croiriez-vous plus facilement à l'avis un expert familiarisé avec les deux types de programmation ou à l'opinion de quelqu'un qui n'a jamais pris le temps d'apprendre l'assembleur et de s'en faire une opinion honnête ?

Dans une conversation, je prendrais de grandes précautions pour poser chacune de ces questions. Certainement, dans une ébauche rigoureuse de ce chapitre j'ai écrit quelque chose comme dix pages en expliquant et en réfutant chacune des mauvaises rumeurs sur l'assembleur. Cependant, ce livre est déjà assez long et j'ai trouvé que passer au crible chacun de ces propos ne serait pas d'un grand avantage. Néanmoins, une courte réfutation de ces mauvaises rumeurs sera utile à convaincre le lecteur que les objections qu'on peut lancer sont plus que légitimes.

L'assembleur est dur à apprendre.

C'est le cas pour tout langage que vous ne connaissez pas. Essayez d'apprendre (mais de vraiment apprendre) APL, Prolog ou Smalltalk. Une fois Pascal appris, apprendre un autre langage tel que C, BASIC, fortran, Modula-2 ou Ada est certainement aisé, parce que tous ces langages se ressemblent et ressemblent à Pascal. D'autre part, apprendre un langage différent comme Prolog est toute une autre affaire. L'assembleur est aussi bien différent de Pascal que de Prolog. D'où sa difficulté. Et cependant, apprendre l'assembleur n'est pas beaucoup plus difficile qu'apprendre votre premier langage de programmation.

L'assembleur est difficile à lire et à comprendre.

C'est sûr, si vous ne le connaissez pas, comme c'est le cas de la plupart qui font cette affirmation. Bien sûr, c'est très facile d'écrire des programmes impossibles à lire en asembleur, mais c'est également très facile de le faire en C, en Prolog ou en APL. Avec de l'expérience vous trouverez l'assembleur aussi facile à lire que les autres langages.

L'assembleur est difficile à déboguer.

C'est la même histoire qu'au point précédent. Ce sera sûrement difficile si vous n'avez pas d'expérience. Vous souvenez-vous des bogues de vos premiers programmes en Pascal (ou d'autres LHN) ? Chaque fois que vous apprenez un nouveau langage vous aurez des problèmes de ce genre, tant que vous ne gagnez pas d'expérience.

[2] Langages de Haut Niveau. De la sigle anglaise HLL, High Level Languages (ndt).

L'assembleur est difficile à maintenir.

Les programmes C sont également difficiles à maintenir. Tous les programmes sont difficiles à maintenir. Les programmeurs inexpérimentés ont la tendance à écrire des programmes de la sorte. Mais écrire quelque chose qui soit maintenable, ce n'est pas un talent. C'est une habileté que vous développez à travers la pratique.

L'assembleur est un langage difficile.

Cette affirmation contient un peu de vérité. Pendant longtemps, les programmeurs en assembleur ont écrit leurs programmes complètement à partir de rien en réinventant souvent la roue. Les utilisateurs des langages de haut niveau, spécialement C, Ada et Modula-2 ont largement bénéficié d'une *bibliothèque standard*, contenant des packages de code servant à résoudre des problèmes communs de programmation. Mais, d'autre part, on a toujours cru les utilisateurs de l'assembleur capables de récrire une routine de sortie d'entiers chaque fois qu'ils en avaient besoin. Ce livre *n'utilise pas* cette approche. Au contraire, il prend plein avantage des travaux effectués à l'Université de Californie Riverside : (*The UCR Standard Library for 80 x 86 Assembly Language Programmers*).

Ces sous-routines simplifient la tâche de programmer tout comme la bibliothèque standard rend plus facile la programmation en C. Les codes source de cette librairie sont disponibles en ligne, ainsi que certains autres services de communication, comme des disquettes d'accompagnement.

La programmation assembleur demande du temps.

Les ingénieurs logiciels estiment que les développeurs emploient à peu près trente-trois pourcent de leur temps à coder la solution d'un problème donné. Même s'il faut deux fois plus de temps pour coder un programme en assembleur qu'un équivalent de haut niveau, il y a seulement quinze pourcent d'écart dans la durée totale d'un projet. En effet, de *bons* programmeurs assembleur n'ont pas besoin d'autant de temps pour produire quelque chose. C'est vrai qu'utiliser un LHN peut faire économiser un *certain* temps ; néanmoins, ce qui est économisé d'un côté ne tient pas le rapport avec ce qu'on gagne de l'autre.

Des compilateurs améliorés ont éliminé le besoin de l'assembleur.

Ceci n'est pas vrai et probablement ne le sera jamais. Certes, l'optimisation des compilateurs est chaque jour meilleure. Cependant, les programmeurs assembleur obtiennent une meilleure performance en écrivant leur code *différemment* par rappor à comment on l'écrirait dans un langage de haut niveau. Certes, si les programmeurs traduisaient manuellement leur code C en assembleur, un bon compilateur les dépasserait sûrement. *Ceux qui soutiennent la supériorité des compilateurs se basent sur la comparaison entre un code traduit à la main et un code produit par un compilateur*. Les compilateurs font un travail de compilation bien meilleur que n'importe quel être humain. Cependant, ne vous attendez pas à trouver des programmeurs assembleur écrivant du code C avec des instructions MOV. D'après tout, c'est justement ceci que les compilateurs font.

De nos jours, les machines sont tellement rapides qu'on n'a plus tellement besoin de l'assembleur.

C'est incroyable ce qu'on dépense en argent en achetant des machines toujours plus rapides, quand, avec le même matériel et un petit investissement de temps supplémentaire on pourrait écrire des programmes plus rapides. Il y a plusieurs débats animés à propos de la vitesse des machines par rapport à celle des logiciels, mais un fait reste certain : les utilisateurs veuleut toujours plus de vitesse. Dans tout ordinateur donné, le programme le plus rapide sera écrit en assembleur[3].

Si vous avez besoin de plus de vitesse, vous devriez utiliser un meilleur algorithme au lieu de passer à l'assembleur.

Et pourquoi ne pas utiliser ce meilleur algorithme en langage assembleur ? Qu'aarive-t-il si vous êtes déjà en train d'utiliser le meilleur algorithme et vous êtes tout de même insatisfait des performances ? Cet argument contre l'assembleur est totalement faux. Tout algorithme que vous pouvez développer dans les LHN vous pouvez l'implémenter en assembleur. D'autre part, il y a nombre d'algorithmes pouvant être implémentés en assembleur qui ne peuvent pas l'être en langage de haut niveau[4].

Les ordinateurs ont tellement de mémoire aujourd'hui qu'économiser de l'espace à l'aide de l'assembleur n'a plus d'importance.

Donnez un mètre à quelqu'un, il lui faudra un kilomètre. Vrai nullepart ailleurs qu'en programmation, surtout pour ce qui concerne la gestion de la mémoire. Les programmeurs se sont contentés longtemps de 4 Ko. Plus tard, les ordinateurs ont disposé de 32 ou même 64 Ko. Et les programmes devaient tenir dans cet espace. Aujourd'hui, beaucoup d'ordinateurs

3 Naturellement, ce n'est pas une règle absolue. Un programme assembleur médiocrement écrit peut tourner beaucoup plus lentement qu'un équivalent de haut niveau. D'autre part, si un programme est écrit dans un langage de haut niveau, il est certainement possible d'en écrire un plus rapide en assembleur.

4 Nous verrons certains de ces algorithmes plus loin dans cet ouvrage. Ils ont affaire à des séquences d'instructions et à d'autres astuces basées sur le fonctionnement spécifique du microprocesseur.

ont 32 ou 64 Mo de mémoire installée et certaines applications l'utilisent entièrement. Il y a beaucoup de raisons techniques à faveur des programmes plus courts, bien que maintenantm il n'est pas encore temps de l'expliquer. Mais, disons simplement que l'espace *est* important et les programmeurs devraient faire l'effort d'écrire des programmes aussi économiques que possible, sans se soucier de la quantité de mémoire qu'ils ont dans leur machine.

Le langage assembleur est, par définition, non portable.

Ce fait est indéniable. Un programme 80x86 écrit pour un IBM PC ne tournera pas sur un Apple Macintosh[5]. Et les programmes écrits pour un Apple Macintosh ne fonctionneront pas sur un Amiga, à dépit du fait qu'il s partagent le même microprocesseur 680x0. Si vous avez besoin de faire tourner vos programmes sur différentes plateformes vous aurez affaire à une longue et dure épreuve. Utiliser C (ou quelque autre LHN) ne garantit pas la portabilité. Des programmes C écrits pour IBM PC ne se compilent pas sur un Macintosh. Et même s'ils le pouvaient, la plupart des utilisateurs Mac n'accepteraient pas les résultats.

La portabilité est probablement le plus grand grief que les gens adressent à l'assembleur. Ils refusent de l'utiliser parce qu'il n'est pas portable, tout en continuant à écrire en C des équivalents non portables.

Oui, il y a beaucoup de mensonges, d'idées fausses, mythes et démi-vérités concernant le langage assembleur. Quoi que vous en pensiez, apprenez l'assembleur avant de vous faire des opinions.

Parler d'un sujet sans le connaître peut tromper ceux qui en savent encore moins que vous, mais n'impressionnera pas ceux qui savent plus.

2 Qu'est-ce qui est vrai au sujet de l'assembleur ?

Une vieille boutade dit quelque chose comme : « Il y a trois raisons pour utiliser l'assembleur : vitesse, vitesse et encore vitesse. » Même ceux qui le détestent vraiment diront que si la vitesse est votre principale préoccupation, alors l'assembleur est la seule voie à suivre. En effet, ses avantages certains sont :

- Vitesse. Les programmes assembleur sont généralement les plus rapides.
- Espace. Les programmes assembleur sont généralement les plus légers.
- Moyens. Vous pouvez faire des choses en assembleur qui sont difficiles ou impossibles dans des langages de haut niveau.
- Connaissances. Votre connaissance de l'assembleur vous aidera à écrire de meilleurs programmes de haut niveau.

L'assembleur est sans conteste le champion de la vitesse parmi les langages. Un expert en assembleur produira presque toujours des programmes plus rapides qu'un expert en C[6]. Même si certains programmes ne tirent pas particulièrement avantage d'une implémentation en assembleur vous pouvez avec un code approprié les rendre de 5 à 10 fois plus rapides que leurs équivalents LHN ; une amélioration encore plus grande est possible si vous n'utilisez pas des compilateurs optimisés. Hélas ! de telles performances sont difficilement réalisables par des programmeurs non expérimentés. Cependant si vous prenez le temps de de bien apprendre l'assembleur, vous pourrez y arriver aussi.

Bien que certains programmeurs considèrent qu'ils n'ont pas vraiment à se préoccuper de la mémoire, il y en a un grand nombre qui auraient besoin d'écrire des programmes plus petits. Et les programmes écrits assembleur occupent souvent la moitié de la taille de leurs équivalents de haut niveau. Ceci est spécialement impressionnant quand vous considérez le fait que les données consomment la même quantité d'espace dans les deux types de programmes, et ces données sont responsables d'une bonne partie de la taille d'une application ordinaire. Economiser de l'espace c'est économiser de l'argent. Purement et simplement. Si un programme requiert 1,5 méga-octets il ne logera pas sur une disquette de 1,44 méga-octets. De façon similaire, si une application requiert 2 méga-octets de RAM, l'utilisateur devra installer un méga-octet supplémentaire si sa machine ne dispose que d'un méga-octet[7]. Même dans des machines de 32 Mo ou plus, écrire des applications gigantesques n'est pas excusable. La plupart des utilisateurs ont désormais plus de 8 Mo de RAM dans leurs ordinateurs, donc ils peuvent exécuter plusieurs programmes à la fois. Plus une application sera gourmande en ressources, moins de programmes on pourra ouvrir simultanément. Et la mémoire virtuelle n'est pas une

5 Strictement parlant, ce n'est pas vrai. Il existe un programme nommé SoftPC qui émule un IBM PC à l'aide d'un interpréteur 80286. Ce qui n'empêche pas qu'à la base, les programmes 80x86 ne fonctionnent pas sur les Mac 680x0.

6 Il n'y a absolument de raison pour qu'un programmeur en assembleur produise du code lent, étant donné qu'il peut toujours inspectionner la sortie produite par le compilateur C et copier les portions de code qui produisent un résultat meilleur que quelque chose d'écrit à la main. Les programmeurs de haut niveau n'ont pas ce privilège.

7 Vous pouvez substituer ici tous les chiffres que vous voulez. Le seul fait qui reste certain est que les programmeurs sont fameux pour considérer que les utilisateurs ont plus d'argent de ce qu'ils ont en réalité.

solution particulièrement attrayante. Avec elle, la taille des programmes est directement proportionnelle à la lenteur du système. Le potentiel est une autre raison pour laquelle les gens ont recours à l'assembleur. Les langages de haut niveau sont une abstraction de la véritable architecture de l'ordinateur. Et ils sont conçus pour être indépendants de cette dernière. En conséquence, ils prennent rarement en compte toute caractéristique spéciale de la machine, caractéristique autrement disponible en assembleur. Si vous voulez disposer de ces possibilités, vous aurez besoin de l'assembleur. Un bon exemple sont les instructions d'entrées/sorties dans les processeurs 80x86. Ces instructions permettent d'accéder directement à certains périphériques d'entrée et de sortie. En général, ces accès ne font pas partie des langages de haut niveau. En effet, certains langages comme C se font un orgueil de ne pas supporter toute opération d'entrée-sortie spécifique[8]. Avec l'assembleur vous n'aurez pas de telles restrictions. Tout ce que vous pouvez faire avec un ordinateur, vous pourrez le faire en assembleur. Ceci n'est définitivement pas le cas des langages de haut niveau.

Bien sûr, une autre raison d'apprendre l'assembleur est la connaissance pure. Maintenant certains d'entre vous pourront penser : "ce serait merveilleux, mais j'ai beaucoup à faire et mon temps serait mieux employé à écrire du code plutôt qu'à apprendre un autre langage". Il y a toutefois de bien pratiques raisons à cette démarche, même si vous ne compterez jamais d'écrire une seule ligne en assembleur. Si vous connaissez bien l'assembleur, vous aurez une idée claire de ce qu'un compilateur fait et vous saurez clairement ce qui se passe réellement avec des instructions de haut niveau. Une fois que vous voyez comment un compilateur traduit des instructions à l'apparence inoffensives dans des tonnes de code machine *vous voudriez sans doute trouver un meilleur moyen de faire la même chose*. *De bons* programmeurs d'assembleur font de meilleurs programmes de haut niveau parce qu'ils comprennent les limitations du compilateur et ils savent ce qu'il fait avec leur code. Ceux qui ne connaissent pas l'assembleur devront tout simplement accepter les piètres performances produites par leur compilateur et hausseront simplement les épaules.

3 Organisation de cet ouvrage et intérêts pédagogiques :

Ce livre est divisé en sept sections principales : une section d'organisation et d'architecture matérielle, une section d'assembleur de base, une section d'assembleur intermédiaire, une section sur les interruptions et sur les programmes résidents, une section consacrée aux particularités de l'IBM PC, une sur l'optimisation et plusieurs appendices. Il est douteux qu'un seul cours d'université (même long) puisse couvrir toute cette matière, les chapitres finaux ont été conçus pour traiter la conception des compilateurs, des micro-ordinateurs, des systèmes d'exploitation et d'autres cours souvent trouvés dans un programme universitaire à part.

Développer un ouvrage comme celui-ci est vraiment une tâche difficile. D'abord, différentes universités ont différentes idées à propos de comment un cours devrait être enseigné. De plus, le temps consacré à ce sujet varie selon les écoles et les besoins (deux trimestres séparés, un semestre unique ou encore un an). Et encore, différentes écoles couvrent différentes matières spécifiques. Par exemple, certaines sont spécialmenent orientées vers le côté « organisation matérielle » et se vaudront des instructions assembleur uniquement pour illustrer des concepts matériels, mais sans prétendre des étudiants la véritable maîtrise de ces instructions (car c'est le compilateur qui s'en occupe). D'autres écoles combinent les deux choses en enseignant l'organisation matérielle et l'assembleur. D'autres encore se dirigent vers l'organisation matérielle et la logique numérique comme une condition requise par un cours d'assembleur. D'autres enseigneront directement l'assembleur en laissant à part le côté de l'organisation matérielle. Finalement, il ne faut pas oublier que certaines personnes apprennent l'organisation matérielle et l'assembleur par eux-mêmes, sans prendre un cours formel sur le sujet. Un bon texte doit donc pouvoir venir en aide aux besoins du cours, du professeur et de l'étudiant. Ces exigences demandent beaucoup à un auteur, mais j'ai prétendu encore davantage de cet ouvrage. Certains livres sont particulièrement clairs sur un sujet, mais une fois la matière lue et maîtrisée, ils n'ont pas une grande utilité en tant que référence. Etant donné le coût des livres aujourd'hui, il est vraiment honteux que beaucoup d'ouvrages perdent leur valeur une fois qu'un cours est terminé. J'ai cherché de créer un livre susceptible d'expliquer différents concepts difficiles d'une façon aussi simple que possible et qui *servira* également comme guide de référence une fois que le sujet aura été maîtrisé. En vous aventurant dans des sujets plus avancés, vous constaterez qu'ils ne seront probablement pas l'objet d'un cours ordinaire d'université, donc vous continuerez d'utiliser cet ouvrage même après la fin de vos études. Par ceci j'espère d'apporter une contribution valide avec ce livre.

Etant donné que ce livre tente de satisfaire à la fois les exigences de différents cours et de fournir une excellente référence, vous trouverez probablement qu'il contient beaucoup plus de matière qu'un cours unique. Par exemple, la première section de cet exposé couvre l'organisation matérielle. Si vous avez déjà abordé le sujet dans un précédent cours, votre professeur peut choisir de sauter les premiers quatre chapitres. Pour les classes enseignant seulement l'assembleur, le professeur peut décider de sauter les chapitres deux et trois. Certains cours de courte durée peuvent

8 Certains langages supportent des extensions sur des périphériques E/S étant donné leurs limitations. Cependant, ces extensions ne font pas partie du langage en tant que tel.

décider de survoler brièvement les chapitres (par exemple, en dédiant une semaine pour chaque chapitre). D'autres écoles pourront traiter la matière avec beaucoup plus de profondeur, parce qu'elles ont plus de temps.

En écrivant cet ouvrage, j'ai décidé de prendre chaque sujet et de le traiter en profondeur avant de passer au suivant. Cette pédagogie (ou technique d'enseignement) est inhabituelle. Beaucoup d'autres ouvrages sautent d'un sujet à l'autre en retournant sur un thème donné au fur et à mesure que l'explication se fait nécessaire. Malheureusement, de tels textes constituent de pauvres références ; essayer de chercher une information dans ces livres est ardu au plus haut degré, parce que chaque sujet est fragmenté tout au long de l'ouvrage. Etant donné que j'ai voulu faire de ce livre aussi une référence correcte, une telle organisation était à proscrire.

Le problème avec une référence rigoureuse serait triple. D'abord de tels manuels sont souvent organisés d'une manière qui rend facile la consultation, mais qui ne suit pas une séquence logique pour ceux qui apprennent[9]. Par exemple, beaucoup de manuels abordent les instructions de l'assembleur par ordre alphabétique. Mais l'ensemble des instructions ne s'apprend pas de cette façon. L'autre problème avec un (bon) manuel de référence est que les informations y sont présentées de façon trop approfondie et dense pour un étudiant. C'est la raison pour laquelle les manuels pédagogiques reviennent sur un sujet abordé pour traiter des nouvelles notions à la lumière ce ce qui a été appris entre temps. Finalement, les textes de référence peuvent présenter la matière dans un ordre quelconque. L'auteur n'a donc pas besoin de s'assurer qu'un sujet traité comprenne seulement des notions déjà vues précédemment. Des matières faisant l'objet des premiers chapitres peuvent se référer à des sujets qui apparaissent dans les derniers. Or, un livre d'université *ne devrait pas* faire ceci.

Pour tirer le mieux de cette lecture, vous aurez besoin de lire et de comprendre son organisation. Il ne s'agit pas d'un ouvrage qu'on lit du début à la fin, en s'assurant de comprendre chaque petit détail d'un chapitre avant de passer au suivant. Certains sujets sont traités dans un détail considérable. Quelqu'un qui apprend l'assembleur pour la première fois pourrait se sentir submergé par la matière qui paraît dans chaque chapitre. Dans ce cas, vous devriez plutôt lire les bases essentielles d'un chapitre pour ensuite y revenir et apprendre la matière additionnelle dont vous avez besoin. Puisqu'il est peu probable de savoir d'avance quel ou quel autre sujet est basique ou avancée, j'ai pris la liberté de l'indiquer. Un cours de dix semaines sur tout le livre, par exemple, pourrait seulement couvrir les notions de base. Dans un cours d'un semestre, ce serait plutôt l'occasion de passer en revue les concepts intermédiaires. Selon les prérequis et la longueur des cours, le professeur peut choisir d'enseigner cette matière dans tout niveau de détail (ou même de naviguer autour du texte).

Dans le passé, si un étudiant terminait une session d'assembleur et était capable d'implémenter un algorithme dans ce langage, le professeur considérait probablement le cours comme parfaitement réussi. Cependant, la technologie des compilateurs a progressé au point que le simple « faire quelque chose que ça marche », est pure folie. Si vous n'arrivez pas à écrire un code efficace en assembleur, tant mieux continuer avec les langages de haut niveau. Ils sont de facile utilisation, et le compilateur générera probablement un code plus rapide que le vôtre, donc il est important de se préoccuper du processus de codage.

Ce livre consacre un grand laps de temps à l'organisation matérielle. Il y a deux importantes raisons à cela. Avant tout, écrire un code efficace sur les processeurs modernes demande une connaissance intime de ce qui se passe dans le matériel. Sans cette connaissance, vos programmes sur des processeurs comme le 80486 ou ultérieur, tourneront au plus à la moitié de la vitesse à laquelle ils pourraient s'exécuter. Pour écrire les meilleurs programmes possibles en assembleur vous devrez avoir une certaine familiarité avec le fonctionnement du matériel. Une autre raison à suffrage de l'importance à avoir une claire idée de l'organisation matérielle est qu'un grand nombre d'universités et de grandes écoles sont de plus en plus portées à mettre l'emphase sur le côté matériel plutôt que sur un langage assembleur particulier. On peut ne pas avoir besoin de connaître l'assembleur pendant les années de préparation à l'université, cependant la portion de l'organisation de la machine est toujours une préparation nécessaire et incontournable, en tant que préparation d'un ou plusieurs cours plus avancés. Des sujets comme les structures de données, les algorithmes, l'architecture matérielle, les systèmes d'exploitation ou la conception des langages et des compilateurs tireront tous parti d'un cours d'introduction à l'organisation matérielle. C'est pourquoi cet ouvrage a consacré une section complète à ce sujet.

4 Obtenir les codes sources des programmes et d'autres outils sur cet ouvrage

9 Notamment, beaucoup de ces manuels manquent d'exemples. Toutes ces outils sont cependant disponibles sur l'internet, n.d.t.

Tous les programmes exemple sont disponibles sur disquette[10]. Le matériel accompagnant ce livre vient en deux parties : des source de divers exemples présentés dans ce livre et le code pour *La bibliothèque Standard de l'UCR pour les programmeurs en Assembleur 80x86*[11]. La bibliothèque UCR est également disponible en format électronique sur diverses sources (en incluant BIX et d'autres services en ligne).

Vous pourrez obtenir les fichiers via ftp de l'adresse internet suivante :

ftp.cs.ucr.edu

Entrez dans ce site, en utilisant un nom d'utilisateur et un mot de passe anonymes. Passez au sous-répertoire "/pub/pc/ibmpcdir" (c'est du UNIX, donc assurez-vous d'utiliser des lettres minuscules). Vous trouverez les fichiers appropriés en cherchant dans ce dossier.

Le(s) nom(s) de fichier(s) exact(s) de ce matériel peut changer avec le temps, et différents services utilisent différents noms pour ces fichiers. Généralement une simple recherche à propos de la bibliothèque UCR générera des réponses appropriées.

10 Ce livre parle souvent de cédérom ou de disquettes d'accompagnement. Ne vous étonnez pas. Ceci était écrit en vue d'une publication de l'ouvrage qui évidemment n'a pas eu lieu, pour des raisons inconnues au traducteur, n.t.d. De plus les liens fournis à la fin de ce chapitre ne sont plsu valides. The original book and resources can still be found here https://www.plantation-productions.com/Webster/

11 UCR Standard Labrary for 80x86 Assembly Language Programmers.